# SSH access from outside your local network

**Recommended route:** ordinary OpenSSH over Tailscale. Both devices join your Tailscale network; you connect to the server's Tailscale address from another Wi-Fi network, mobile data, or another location. No router port forwarding is needed. Your normal Linux account and SSH key remain the login credentials.

**Alternative:** public IPv4 plus a router TCP forwarding rule. This requires control of the router and a usable public IPv4. CGNAT, upstream NAT, and some managed networks prevent this route. A public IPv6 connection is another option, with separate firewall requirements described below.

This guide targets **Arch Linux with systemd**. The helper also supports ordinary Debian/Ubuntu and Fedora daemon-service installations. The SSH client can run on Linux, Windows, macOS, or Android. Windows is covered as a client, not a server.

## 1. What the address and credentials actually mean

| Item | What you use | How to obtain it |
|---|---|---|
| Server | The Linux computer receiving the connection | Your target computer |
| Client | The device from which you connect | Laptop, desktop, or phone |
| Username | Existing Linux account, e.g. `heini` | `id -un` on the server |
| Tailscale address | Server's Tailscale IP or MagicDNS name | Helper or `tailscale ip -4` |
| Public IPv4 | Router's WAN IP, if publicly routable | Router WAN page; compare with egress IP |
| LAN address | Server's address inside the router's network | Helper or `ip -br address` |
| Server port | Port on which `sshd` listens; default `22` | Helper's selected `--port` |
| External IPv4 port | Port forwarded by the router; example `2222` | Your forwarding rule |
| Authentication credential | Private key **on the client** | Generate it on the client |
| Key passphrase | Passphrase protecting that private key | Set it yourself; it cannot be recovered |
| Server public key | Matching `.pub` line in `authorized_keys` | Copy the client's public key to the server |
| Host fingerprint | Identity of the server, not the client | `ssh-keygen -lf` on server host public keys |

The helper can discover addresses, usernames, service status, and fingerprints. It cannot retrieve passwords or key passphrases, infer the router's actual WAN address from a private LAN address, or prove that a router forwards incoming connections. These require information or a test from the relevant device.

## 2. Fast setup with the attached helper

Keep `ssh-remote-helper.py` and this guide together. Substitute your actual Linux username if it is not `heini`. Use a local terminal on the server for initial configuration and retain that terminal until a new client login succeeds.

### 2.1 Install the server prerequisites on Arch

```
sudo pacman -Syu --needed openssh iproute2 python python-rich tailscale
```

`openssh` supplies the daemon, client, and key tools. `iproute2` supplies network discovery. Rich supplies the formatted terminal tables; the script works without Rich using plain text. The full Arch update avoids a partial package upgrade.

For public access without Tailscale, omit `tailscale` from that command.

### 2.2 Generate a key on the CLIENT, without sudo

On a Linux client with Python and OpenSSH:

```
python3 ssh-remote-helper.py client-key
```

This creates `~/.ssh/id_ed25519_remote` and its `.pub` file. Set a strong passphrase when `ssh-keygen` asks. Existing key pairs are reused rather than overwritten. You can choose a different path:

```
python3 ssh-remote-helper.py client-key \
  --key-path "$HOME/.ssh/id_ed25519_home"
```

Copy **only the `.pub` file** to the server using a USB drive, an existing trusted connection, or by pasting the displayed public key into a file. For example, save the public line on the server as `/home/heini/client-key.pub`.

The private key stays on the client. Generating it on the server would give you the wrong starting point for authenticating your client.

### 2.3 Preview on the SERVER

```
python3 ssh-remote-helper.py setup \
  --user heini \
  --public-key /home/heini/client-key.pub \
  --mode tailscale
```

The preview shows the configuration and available network details. It does not change SSH files or start services. Public IP discovery uses two HTTPS services as fallbacks; add `--no-public-ip` to skip these lookups.

### 2.4 Apply on the SERVER

```
sudo python3 ssh-remote-helper.py setup \
  --apply \
  --user heini \
  --public-key /home/heini/client-key.pub \
  --mode tailscale
```

If dependencies are missing, `--install` allows their interactive installation. On Arch, that can run a full `pacman -Syu` update.

The helper starts `tailscaled`, requests browser authentication if needed, and uses your existing Tailscale login if already connected. Authenticate with the account used by your other device, or a tailnet that grants that device access.

This setup uses **normal OpenSSH over Tailscale**. Tailscale's separate SSH feature uses different authentication. If already enabled, the helper stops and shows this command for intentionally

switching back to normal OpenSSH:

```
sudo tailscale set --ssh=false
```

It then installs your public key, configures key-only login, enables the normal SSH service, checks the effective daemon settings, and checks a local SSH banner. It prints a recovery backup path and the remote connection command.

### 2.5 Connect Tailscale on the CLIENT

On an Arch client:

```
sudo pacman -Syu --needed openssh tailscale
sudo systemctl enable --now tailscaled.service
sudo tailscale up
tailscale status
```

Sign into the same permitted Tailscale network. The tailnet's network access policy must allow the client to reach the server's selected TCP port. For this ordinary OpenSSH setup, the relevant policy is network access to that port; Tailscale SSH-specific login rules are not what authenticates you.

Use the **actual command printed by the server helper**. A fictional example:

```
ssh -i ~/.ssh/id_ed25519_remote -o IdentitiesOnly=yes \
  -p 22 heini@100.101.102.103
```

That example IP is a placeholder. Compare the server host fingerprint in the client's first connection prompt with the fingerprint printed in the trusted server terminal before accepting it. `ssh-keyscan` alone is not identity proof.

The helper's printed command includes `-i ~/.ssh/id_ed25519_remote` by default, matching the key created by `client-key`. For another key, supply its path on the **client** with `--client-identity '~/.ssh/id_ed25519_home'` during setup or inspection. The server never opens that private-key path. Quoted `~` in the printed identity argument is expanded by OpenSSH itself.

### 2.6 Apply the relevant host firewall rule

The helper prints UFW and, if active, firewalld commands. Execute the commands for **the firewall you actually use**. It does not reset or enable a firewall.

For an existing UFW installation, permitting SSH through Tailscale:

```
sudo ufw allow in on tailscale0 proto tcp to any port 22
sudo ufw status verbose
```

This adds a permit rule. It does not remove an existing rule allowing SSH from everywhere. Audit those existing rules if access should be limited to Tailscale. Tailscale also manages networking rules of its own; inspect the actual effective input policy rather than assuming a single UFW line describes everything.

For firewalld, use the helper's generated rich rule for the detected zone. The rule limits its permission to the server's Tailscale IPv4 and Tailscale source range. The helper prints both a runtime and a permanent command; a reload is unnecessary after adding both. Existing broader zone permissions still matter.

For native nftables, inspect your rules and add an interface/port permit to your existing input chain before its final rejection:

```
sudo nft list ruleset
# Rule expression for your EXISTING input chain:
# iifname "tailscale0" tcp dport 22 accept
```

The chain/table name is installation-specific. Creating a second arbitrary base chain is not a reliable replacement for editing the existing firewall.

**The daemon listens on local IPv4 and IPv6 interfaces.** Tailscale provides the remote network path; it does not close ordinary LAN or public-interface SSH listeners. Avoid router forwarding for this route and use the host firewall to control other interfaces, especially on public Wi-Fi or a directly addressed IPv6 interface.

## 3. Public IPv4 and router port forwarding

Use this when you control the network and want direct SSH without a client VPN. On a hospital, university, hotel, or other managed network, router administration is generally unavailable. Use Tailscale when permitted by that network.

### 3.1 Discover the network details

On the server:

```
ip -br address
ip -4 route
python3 ssh-remote-helper.py inspect --user heini --mode public
```

The helper labels the public IP lookup as **observed egress IPv4**. That is the address seen by an external HTTPS service. A VPN, proxy, or exit node can make it different from the address to which incoming connections should be sent.

Open your router's **Internet/WAN status** page and find its WAN IPv4. Then:

```
python3 ssh-remote-helper.py inspect \
  --user heini --mode public --router-wan ACTUAL_ROUTER_WAN_IPV4
```

Replace `ACTUAL_ROUTER_WAN_IPV4` with the real value; the helper validates it.

| WAN result | Interpretation and next step |
|---|---|
| `100.64.0.0/10` | Shared address space; likely CGNAT. Request public IPv4 or use Tailscale. |
| Private WAN such as `192.168.x.x` or `10.x.x.x` | Upstream NAT. Configure the upstream router too, if you control it. |
| Public WAN matches observed egress | Public IPv4 is plausible; forwarding and inbound testing remain necessary. |
| Public WAN differs from observed egress | Investigate VPN routing, proxies, upstream networking, or a stale lookup. |
| Router WAN is unavailable | The helper cannot establish whether CGNAT is present. |

Dynamic DNS solves changing public addresses. It does **not** bypass CGNAT. The Tailscale address also uses `100.64.0.0/10`; seeing that on `tailscale0` is normal and says nothing about the router's WAN address.

### 3.2 Configure the server

```
sudo python3 ssh-remote-helper.py setup \
  --apply \
  --user heini \
  --public-key /home/heini/client-key.pub \
  --mode public \
  --port 22 \
  --external-port 2222 \
  --endpoint YOUR_REAL_PUBLIC_IP_OR_DNS_NAME
```

Use your actual public IP or DNS name, without `https://` or a port suffix. Omitting `--endpoint` uses the observed egress IP as a **candidate**, not a proven working address. Add `--router-wan` if you have its value.

`--external-port` changes the reported router/client port, not the server's listener. `--port` changes the actual server listener. Retaining server port 22 while using external port 2222 usually requires fewer local configuration changes. A different port reduces some automated noise; key authentication and firewall policy provide the meaningful security controls.

### 3.3 Reserve the server's LAN IP and forward TCP

In the router, create a DHCP reservation for the server's LAN network interface. Use the correct interface/MAC address; the helper lists network interfaces and addresses. You can inspect the MAC addresses with:

```
ip -br link
```

Example values below are fictional:

| Router setting | Example |
|---|---|
| Protocol | TCP |
| External/WAN port | 2222 |
| Internal destination | 192.168.1.50 |
| Internal destination port | 22 |
| Rule status | Enabled |

Do not forward UDP or put the server in a router DMZ. With double NAT under your control, the upstream router must forward to the downstream router's WAN address and the downstream router must forward to the server.

### 3.4 Allow the server port through the host firewall

For UFW with a fixed, trusted client public IPv4, prefer a source restriction:

```
sudo ufw allow proto tcp from CLIENT_PUBLIC_IPV4/32 to any port 22
sudo ufw status verbose
```

With a changing mobile/client address, a fixed source restriction will stop working when that address changes. Tailscale is usually a better fit. If you choose unrestricted public key-only access with existing UFW:

```
sudo ufw allow 22/tcp
```

The host permits **22**, while the router permits/forwards **2222** in this example. Use `--source-cidr CLIENT_PUBLIC_IPV4/32` to have the helper generate restricted UFW/firewalld commands. Inspect existing broader rules as well.

Cloud/VPS servers also need their provider's ingress/security-group rule. A host firewall permission alone does not open a provider firewall.

### 3.5 Test from a genuinely different network

Disable Wi-Fi on your phone and use mobile data, or connect through another network. From the client:

```
ssh -i ~/.ssh/id_ed25519_remote -o IdentitiesOnly=yes \
  -o ConnectTimeout=10 -p 2222 heini@YOUR_REAL_PUBLIC_IP_OR_DNS_NAME
```

A same-LAN test against the public address depends on router NAT loopback and can succeed or fail independently of off-site access. A public IP lookup is also an outbound test, not an inbound test.

## 4. Direct public IPv6

With working IPv6 on both devices, use the **server's global IPv6**, rather than the router's IPv6 address. Link-local addresses (`fe80::/10`) and local addresses (`fc00::/7`) are not public Internet destinations.

```
ip -6 address show scope global
```

Inspect the addresses carefully: `scope global` can include local IPv6 prefixes. Permit the server's actual SSH port through the router's IPv6 ingress firewall and the host firewall. Conventional IPv4 port forwarding is unnecessary here.

```
# Documentation IPv6 below is a placeholder; use your server's actual address.
ssh -6 -i ~/.ssh/id_ed25519_remote -p 22 heini@2001:db8::50
```

In public mode, the helper accepts a literal IPv6 `--endpoint` and reports the **server port** for that IPv6 command. For an IPv6-only DNS hostname, set `--external-port` equal to `--port` for the reported command, or simply use `ssh -6 -p SERVER_PORT`. The helper cannot infer a router mapping from DNS.

Use a stable server IPv6 address/DNS record; temporary privacy addresses and ISP prefix changes can invalidate a saved literal address. Audit IPv6 ingress even if you have never created an IPv4 forwarding rule.

## 5. Make the client command convenient

On a Unix client, add an entry to `~/.ssh/config`. On Windows OpenSSH, the corresponding file is `$HOME\.ssh\config`. Replace the host with the actual address printed by the helper:

```
Host home-linux
  HostName 100.101.102.103
  User heini
  Port 22
  IdentityFile ~/.ssh/id_ed25519_remote
  IdentitiesOnly yes
  ServerAliveInterval 30
  ServerAliveCountMax 3
```

For public IPv4, change `HostName` to your public IP/DNS and `Port` to the external forwarded port. For an IPv6 literal, use the unbracketed address in `HostName`. Then:

```
ssh home-linux
sftp home-linux
scp ./file.txt home-linux:~/
```

You can optionally load the client key into an existing SSH agent:

```
ssh-add ~/.ssh/id_ed25519_remote
```

The agent must already be running. `ServerAliveInterval` detects a dead connection; it does not keep a suspended or powered-off server reachable.

### Windows PowerShell client

If `ssh` and `ssh-keygen` are unavailable, install the Windows OpenSSH Client optional feature from an elevated PowerShell:

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

In your ordinary, non-elevated PowerShell, generate the key:

```
New-Item -ItemType Directory -Force "$HOME\.ssh" | Out-Null
ssh-keygen -t ed25519 -a 100 -f "$HOME\.ssh\id_ed25519_remote"
Get-Content "$HOME\.ssh\id_ed25519_remote.pub"
```

Copy that `.pub` file/line to the server. Install and connect the Tailscale Windows client for Tailscale access. Connect with:

```
ssh -i "$HOME\.ssh\id_ed25519_remote" -p 22 heini@SERVER_TAILSCALE_IP
```

### Android client with Termux

Install and connect the Tailscale Android app for the recommended route. In Termux:

```
pkg install openssh
mkdir -p ~/.ssh
chmod 700 ~/.ssh
ssh-keygen -t ed25519 -a 100 -f ~/.ssh/id_ed25519_remote
cat ~/.ssh/id_ed25519_remote.pub
ssh -i ~/.ssh/id_ed25519_remote -p 22 heini@SERVER_TAILSCALE_IP
```

Install that phone's public key on the server first. Each client can have its own key. To add another key after initial setup, copy its `.pub` file to the server and reapply with the same username, mode, and ports. Existing keys are preserved. Each reapplication creates its own recovery backup.

## 6. Inspect, customize, and restore

### Available commands

```
python3 ssh-remote-helper.py --help
python3 ssh-remote-helper.py --long-help
python3 ssh-remote-helper.py setup --help
python3 ssh-remote-helper.py inspect --user heini
```

| Option | Purpose |
|---|---|
| `--user` | Existing Linux SSH login account; default is caller or sudo caller |
| `--mode tailscale` | Recommended overlay-network path; default |
| `--mode public` | Generate public IP/router forwarding connection details |
| `--public-key FILE.pub` | Public key generated on the client; required to apply |
| `--port N` | Actual server SSH TCP port; default 22 |
| `--external-port N` | Router/client IPv4 TCP port; default 2222 |
| `--endpoint IP_OR_DNS` | Override public connection address |
| `--client-identity CLIENT_PATH` | Private-key path used in printed client commands; default `~/.ssh/id_ed25519_remote` |
| `--router-wan IPV4` | Compare router WAN IPv4 with observed egress |
| `--source-cidr CIDR` | Restrict generated public-mode firewall permissions |
| `--service ssh / sshd` | Select a standard daemon service explicitly |
| `--install` | Permit interactive installation of missing server dependencies |
| `--apply` | Perform setup or restoration; absent means preview |
| `--no-public-ip` | Skip external HTTPS address lookups |
| `--report FILE.json` | Save connection information locally with mode 600 |
| `client-key --key-path PATH` | Choose the client's private-key output path |
| `restore --backup PATH` | Preview restoring the backup printed by setup |
| `restore --backup latest --apply` | Restore the most recent helper backup |
| `restore --force` | Allow replacing tracked files changed after that setup |

Example report on the server:

```
sudo python3 ssh-remote-helper.py inspect \
  --user heini --mode tailscale \
  --report /home/heini/ssh-connection-details.json
```

A report written through sudo is root-owned. Read it with sudo, or intentionally change ownership for your own use:

```
sudo chown heini: /home/heini/ssh-connection-details.json
```

Reports contain network/account details, not passwords or private keys. An inspection report labels its settings as requested values; use the daemon commands below to verify any existing configuration independently.

### Files and service settings changed by setup

The helper adds an early `Include /etc/ssh/remote-helper.conf` to the main configuration, retains the rest of the original file, and writes its managed settings to that separate file. Early placement matters because many SSH settings use the first obtained value. Some settings, including ports and user allow lists, accumulate; the helper checks the resulting effective values.

It appends the selected client's public key to the selected user's `~/.ssh/authorized_keys`, sets that

file to `600`, and sets `.ssh` to `700` with the correct owner. It retains other existing keys and creates missing server host keys without replacing existing host identity keys.

The managed settings permit only the selected non-root account, require a public key, and disable password and keyboard-interactive login. They retain OpenSSH's current cryptographic defaults. TCP forwarding remains enabled for useful workflows such as a local tunnel to an RStudio or Jupyter service.

For example, with a service listening on server localhost port 8787:

```
ssh -N -L 127.0.0.1:8787:127.0.0.1:8787 home-linux
```

Open `http://127.0.0.1:8787` **on the client**. The application still requires its own authentication. This does not require a public application port.

### Existing specialized SSH installations

This helper is deliberately scoped to a personal machine. It refuses existing `Match` policies, conflicting active ports/listeners/user restrictions, custom daemon command-line options, and socket activation. If it detects a conflict after writing, it restores the tracked SSH files and prior service state.

For an Ubuntu installation using `ssh.socket`, convert to the ordinary daemon from a **local console** before using the helper:

```
sudo systemctl disable --now ssh.socket
sudo systemctl stop ssh.service
sudo systemctl enable --now ssh.service
```

Do not perform that conversion through your sole remote SSH connection. On other installations, inspect which socket/service actually exists:

```
systemctl list-unit-files 'ssh*'
systemctl cat ssh.service
systemctl cat sshd.service
```

The helper does not bypass an existing conditional access policy, configure SELinux for a custom port, or rewrite a custom nftables policy. On Fedora with SELinux, keeping server port 22 and changing only the router's external port avoids the extra custom-port SELinux configuration.

### Restore a helper backup

Use the exact backup path printed during the setup you want to undo:

```
sudo python3 ssh-remote-helper.py restore --backup /ACTUAL/PRINTED/BACKUP/PATH
sudo python3 ssh-remote-helper.py restore \
  --backup /ACTUAL/PRINTED/BACKUP/PATH --apply
```

Or, for the latest setup:

```
sudo python3 ssh-remote-helper.py restore --backup latest
sudo python3 ssh-remote-helper.py restore --backup latest --apply
```

Restoration covers the main/managed SSH files, the previous `authorized_keys`, the previous `.ssh` directory permissions, and SSH service active/enabled state. It refuses to overwrite tracked file contents changed since the selected setup unless you explicitly pass `--force` after reviewing those changes. Choose the backup by path when several setups have occurred.

Package installation, generated host keys, Tailscale installation/login/service state, and manually applied router/firewall rules are outside this restoration. Remove a router forwarding rule in the

router when it is no longer wanted. For a UFW rule added exactly as above, the corresponding removal is:

```
sudo ufw delete allow in on tailscale0 proto tcp to any port 22
# Or, for the unrestricted public-mode example:
sudo ufw delete allow 22/tcp
```

## 7. Native CLI verification and troubleshooting

On Arch:

```
sudo sshd -t
sudo sshd -T -C user=heini,host=localhost,addr=127.0.0.1
systemctl status sshd.service --no-pager
sudo journalctl -u sshd.service -n 50 --no-pager
sudo ss -lntp
```

On Debian/Ubuntu, the unit is normally `ssh.service`. Use the detected name from the helper. `sshd -T` reports settings, while `ss` reports actual listening sockets; a daemon launched with command-line overrides can differ from a simple configuration-file inspection.

From the client:

```
ssh -vvv -i ~/.ssh/id_ed25519_remote -o IdentitiesOnly=yes \
  -o ConnectTimeout=10 -p 22 heini@ACTUAL_SERVER_ADDRESS
```

For Tailscale network diagnostics:

```
tailscale status
tailscale ping SERVER_TAILSCALE_IP
tailscale netcheck
```

Tailscale ping checks the overlay path, not successful SSH user authentication.

| Symptom | Most likely checks |
|---|---|
| Timeout | Address, host awake, Tailscale connection/policy, ingress firewall, forwarding, CGNAT |
| Connection refused | SSH service inactive, wrong destination port, explicit firewall rejection |
| `Permission denied (publickey)` | Correct Linux user; client's matching `-i` key; installed `.pub`; permissions; server logs |
| Too many authentication failures | Add `-o IdentitiesOnly=yes` and the intended `-i` key |
| Passphrase prompt | Local private key needs unlocking; this is not the server password |
| Host key changed warning | Verify host identity locally; do not automatically discard the warning |
| Public access fails only from home LAN | Router NAT loopback may be absent; test from mobile data |
| Address worked yesterday | ISP IP/prefix changed; update DDNS, or use the Tailscale address |
| Works until the lid closes | Server suspended; inspect its actual power policy |
| Tailscale works but SSH fails | Normal SSH key/service/firewall issue, or Tailscale SSH interception on port 22 |

The helper does not modify power/lid settings. The machine must remain powered, connected, and awake. After setup, restart it when convenient and repeat a new off-site connection to confirm boot-time service and network availability.

## 8. Other Linux distributions

Debian/Ubuntu prerequisites:

```
sudo apt update
sudo apt install openssh-server iproute2 python3 python3-rich curl
```

Fedora prerequisites:

```
sudo dnf install openssh-server openssh-clients iproute python3 python3-rich curl
```

Install Tailscale using its current official Linux instructions. If using its official installer, download and inspect it before running it:

```
curl -fsSL https://tailscale.com/install.sh -o /tmp/tailscale-install.sh
less /tmp/tailscale-install.sh
sh /tmp/tailscale-install.sh
```

That installer may ask for sudo. The helper's `--install` does not add third-party Tailscale repositories on Debian/Fedora automatically. Its Python code targets Linux; use native `ssh-keygen` on Windows rather than running this helper there.

## 9. Sources

Checked against current documentation on 14 October 2026:

- OpenSSH server configuration
- OpenSSH daemon validation and keys
- OpenSSH key generation
- OpenSSH client configuration
- ArchWiki: OpenSSH
- Tailscale Linux installation
- Tailscale CLI
- Tailscale server setup
- Tailscale SSH and its distinct authentication
- UFW manual
- firewalld rich rules
- Shared address space, RFC 6598
- Microsoft: OpenSSH installation

The helper's backup, reporting, and conservative refusal behaviour are specific to the attached implementation; they are not OpenSSH defaults.